REAL NETWORK TRAFFIC ANALYSIS USING WIRESHARK

Protocol Distribution, Packet Size, and Retransmissions

Course: Data Communication and Networks

Assignment Type: Research-Based Assignment

Tool Used: Wireshark 4.6.4 | Protocols Observed: ICMP, ARP, TCP, ICMPv6, MDNS

Authors: Aarzoo Gupta (B1) and Aegle (B2)

ABSTRACT

This report presents a detailed examination of network traffic behavior which has been done by analyzing a trace captured using Wireshark consisting of 280 packets within a controlled, host-only VirtualBox environment. It captures a span of roughly 637 seconds where we have established bidirectional communication between a guest virtual machine created using VirtualBox (IP: 192.168.198.3) and its host gateway (IP: 192.168.198.1). Our primary goal emphasized in this experiment was to accurately quantify protocol distribution, evaluate round-trip latencies, and carefully observe transmission anomalies at the frame level which lead to retransmissions. We found that the capture consists overwhelmingly of ICMP echo requests and replies, making up 90.7% of the total packets. Beneath this surface layer, we also identified the expected background noise of Address Resolution Protocol (ARP) cache updates, an ICMPv6 router solicitation triggered by operating system initialization, and Multicast DNS (MDNS) printer discovery queries. The most interesting finding was our capture of a totally failed Secure Shell (SSH) connection attempt. This failure resulted in a sequence of repeated TCP SYN retransmissions which perfectly illustrated the standard TCP exponential back-off mechanism algorithm. By looking at these raw, unprocessed packets, we managed to measure actual network performance metrics including throughput, protocol overhead, and packet jitter. These quantitative findings help us assess the performance of the network created. Ultimately, this report, packet-level analysis gives us a reliable baseline for understanding how hosts communicate in isolation, and exactly how TCP connection fails, how are retransmissions initiated and what is the protocol distribution.

1. PROBLEM STATEMENT

1.1 Motivation

Modern computer networks generate various types of traffic which often coexist within the same observation window. As per this, traffic analysis cannot be limited to only the packets intentionally created by a user. Instead, meaningful interpretation requires inspection of all protocols observed on the wire, including those produced automatically by the operating system or network stack, as well as the number of retransmissions that ultimately affect the performance.

We have examined this condition in a small but realistic virtualized environment. The topology contains a guest virtual machine and a host machine that serves as two nodes and it captures multiple layers of network operation: reachability testing through ICMP, address resolution through ARP, a transport-layer connection attempt through TCP SYN retransmissions, IPv6 control behaviour through Router Solicitation, and multicast printer discovery through MDNS. The core problem addressed in this report is therefore not merely how to read packets, but how to transform packet observations into a coherent explanation of network behaviour. Such interpretation is essential because raw frames, by themselves, do not explain whether a network is healthy, overloaded, misconfigured, or silently filtering communication. A structured study of the capture is required to connect packet evidence with performance and operational meaning.

1.2 Significance

The crux of the experiment lies in its detailed examination of performance, diagnostics and security assessment. When an administrator measures round-trip time, investigates retransmissions, or studies protocol distribution, the purpose is not descriptive analysis only, the deeper objective is to infer the performance of the network and understand the protocol distribution with respect to the packets captured. In the capture, sub-millisecond ICMP latency suggests efficient local communication, while the absence of packet loss indicates a stable virtual path. However, repeated unanswered TCP SYN frames reveal a connectivity problem at the service layer despite the fact that basic reachability remains intact. This distinction is operationally important because it demonstrates that successful ping responses do not always guarantee application availability. Similarly, ARP cycles reveal normal network maintenance, whereas ICMPv6 and MDNS frames expose background operating-system behaviour that would be invisible to an analyst relying solely on high-level tools. Accordingly, the report shows how packet capture aids the process of

analyzation of protocols and helps analysts interpret the difference between expected control traffic and potentially suspicious background activity.

1.3 Real-World Relevance

The problem studied in this report has clear real-world relevance beyond the descriptive analysis. Network Operations Centres routinely use packet traces to validate latency, identify abnormal retransmission patterns and distinguish between routing issues and host-level service failures. Security analysts use the same evidence to detect scanning, silent firewall drops, or background services that may enlarge the attack surface. Cloud and virtualization engineers apply packet analysis when troubleshooting east-west traffic between virtual machines, gateways, and hypervisor-managed interfaces. Even in educational settings, small captures such as the present one are valuable because they reveal the coexistence of multiple protocols within a short trace, enabling students to see how network layers interact in practice. This report therefore addresses a broadly applicable question: *how can packet-level evidence from a compact, controlled capture be used to produce disciplined conclusions about protocol composition, timing, service accessibility, and the operational posture of a networked system?*

2. OBJECTIVES

The objectives of this Wireshark-based assignment are listed below:

- To identify and quantify the protocol distribution of the captured traffic, with particular attention to the relative shares of ICMP, ARP, TCP, ICMPv6, and MDNS.
- To determine the packet-size characteristics associated with the major observed protocols.
- To measure and interpret the RTT of the two ping sessions so that latency, responsiveness, and link stability within the host-only virtual network can be assessed.
- To analyze the sequence of TCP SYN retransmissions directed toward port 22.
- To evaluate additional performance indicators, including throughput, jitter, packet loss, and protocol efficiency to present a complete view of network behavior within the experiment.

3. LITERATURE REVIEW

The literature base associated emphasizes three broad themes: the usefulness of packet sniffing for protocol visibility and troubleshooting; the role of packet inspection in security and forensic investigation; and the need to extend descriptive packet captures into more rigorous quantitative analysis. Many studies demonstrate that the tool can capture traffic, decode fields, and expose suspicious behaviour, yet comparatively fewer studies translate those captures into structured metrics such as packet distribution, retransmission rates, timing intervals, jitter, or protocol efficiency. This report is positioned within that gap. Rather than treating Wireshark only as a visibility tool, the document uses the network capture to interpret protocol counts, packet sizes, round-trip timing, and control-plane overhead in a systematic manner.

Table 1. Summary of 15 reviewed research papers.

Paper	Method Used	Result / Contribution	Limitation
Praful Saxena, Sandeep Kumar Sharma (2017) Analysis of Network Traffic by using Packet Sniffing Tool: Wireshark	Created a network between two nodes via switch and used Wireshark to capture network traffic and analysed traffic using packets list, details and bytes panel.	Proves that Wireshark can capture and analyse all types of network packets. Packet sniffing can help detect suspicious activities and reveal sensitive data. TCP session analysis is clearly observed. (3-way handshake)	Very basic and theoretical analysis, no statistical analysis. No use of metrics like retransmission rate and packet size distribution. Thus, no real performance evaluation.
R Soepeno - CYBV-Introd. Methods Netw. Anal, 2023 Wireshark: An effective tool for network analysis	Live packet capture using Wireshark HTTP stream analysis (request/response) Evaluation based on Confidentiality, Integrity, Availability Comparison with TCPdump and NetFlow	Wireshark provides detailed packet analysis, helps to detect security threats, performance measure and shows that HTTPS ensures confidentiality.	No quantitative analysis and scope is restricted to HTTP only. Explains concepts and lacks practical examples. There is no real dataset analysis.
B Dodiya, UK Singh - International Journal of Computer Applications, 2022 Malicious Traffic analysis using Wireshark by collection of Indicators of Compromise	Analysis of .pcap files using Wireshark Application of filters like http, request, nbns, dhcp Verification of malware using VirusTotal	Successfully identified: infected file and its hash malicious domain and IP host details (MAC, hostname) Demonstrated that Wireshark can detect malware activity and support network forensics	No quantitative traffic analysis (no protocol %, packet size, retransmissions) Focus limited to malware detection Uses predefined dataset (not real-time large network)
Network forensics analysis using Wireshark V Ndatinya, Z Xiao, VR Manepalli	Packet capture and analysis using Wireshark Examination of protocols (TCP, HTTP, DNS, etc.) Reconstruction of sessions (e.g., TCP	Wireshark helps us to do packet level inspection, identify unusual activities, reconstruction of user activities like files and sessions.	Focuses on network forensics (after attack) not real time detection. No quantitative metrics used and performance issues

	streams) Identification of suspicious patterns and attack traces		faced when network increases.
R. Tuli (2023) – Analyzing Network Performance Parameters using Wireshark	Packet capture using Wireshark, analysis of network fluctuations and graph-based analysis of traffic vs time.	Helps us to identify the time at which network traffic rises or peaks and drops. Analysis of network parameters and effective bottlenecks	Limited depth (no retransmission rate or protocol distribution) Mostly visualization based. Lacks packet level analysis
S. Kakuru (2011) – Behavior-Based Network Traffic Analysis Tool	Capturing traffic using sniffers and analyse user behaviour patterns like browsing	Helps to distinguish between usual and abnormal behaviour of user activities and mainly used for anomaly detection	High-level analysis but not packet-level, lacks quantitative evaluation
C. Sanders (2017) – Practical Packet Analysis	It ponders around a case (case-based analysis), packet inspection and protocol analysis	It demonstrates practical techniques for network debugging, analysing protocols and packet inspection.	It does not have any experimental analysis, focuses on conceptual learning and no statistical analysis.
P. Chaudhary et al. (2023) – Network Traffic Analysis using Wireshark	Capturing packets using Wireshark, protocol-based filtering and analysis	Serves the purpose of identifying network traffic and useful for real time monitoring	Basic analysis No deep metrics (packet size, retransmissions, etc.)
Jain & Anubha – Application of Snort and Wireshark in Network Traffic Analysis	Packet capture using Wireshark Intrusion detection using Snort (rule-based alerts) Analysis of suspicious traffic patterns	Wireshark provides detailed packet-level inspection Snort detects attacks (e.g., scanning, intrusion attempts) Combined use improves network security monitoring	Focus on security, not traffic metrics No quantitative analysis (protocol %, packet size, retransmissions)
Mabsali, Jassim & Mani (2023) Effectiveness of Wireshark Tool for Detecting Attacks and Vulnerabilities in Network Traffic	Packet capture and inspection using Wireshark Analysis of suspicious traffic patterns Identification of vulnerabilities and attack traces	Wireshark can detect: abnormal traffic patterns potential attacks and vulnerabilities Effective for network monitoring and security analysis	Passive tool (cannot prevent attacks) No quantitative metrics (protocol %, packet size, retransmissions)
Shimonski (2013) The Wireshark Field Guide: Analyzing and Troubleshooting Network Traffic	Real-world examples and case-based learning Packet inspection and protocol analysis	Helps in: diagnosing network issues understanding protocols troubleshooting connectivity problems	Not a research study No experimental or statistical analysis
Kumar & Yadav – "Comparison: Wireshark on Different Parameters"	Comparative analysis of Wireshark with other packet sniffing tools Evaluation based on parameters such as: performance usability (GUI vs CLI)	Wireshark performs strongly in: user-friendly interface (GUI) wide protocol support powerful filtering and detailed packet analysis	No real traffic dataset analysis No quantitative metrics (protocol %, packet size, retransmissions) Focus is tool comparison, not traffic behavior

	protocol support filtering capability	More suitable for beginners and detailed inspection compared to command-line tools	
Allman, M., Paxson, V., & Stevens, W. R. (1999). TCP Congestion Control. RFC 2581, IETF.	Specification of TCP slow-start, congestion avoidance, fast retransmit, and fast recovery algorithms; formal definition of RTO computation.	Defined the exponential back-off rule for SYN retransmissions: RTO doubles on each failure starting at 1 second. This directly explains the observed retransmission timing in packets 84-98 (1s, 2s, 4s, 8s intervals).	RFC defines standard behaviour; OS implementations (Linux, Windows) may deviate in SYN-specific retry counts and initial RTO values, affecting exact timing observed.
Dreger, H., Feldmann, A., Paxson, V., & Sommer, R. (2004). Operational Experiences with High-Volume Network Intrusion Detection. ACM CCS, pp. 2-11.	Deployment and evaluation of Bro IDS on high-speed links; analysis of protocol parser performance and false-positive rates for anomaly detection.	Showed that repeated SYN packets without SYN-ACK responses (as in our capture) are reliably flagged as port scan events by signature-based IDS. Provided baseline detection rates.	High-speed backbone context; the same heuristic applied to a two-host virtual network generates false positives if legitimate SSH retry behaviour is not tuned out.
Roughan, M., Sen, S., Spatscheck, O., & Duffield, N. (2004). Class-of-Service Mapping for QoS: A Statistical Signature-Based Approach to IP Traffic Classification. ACM IMW.	Machine-learning classification of IP traffic flows by protocol using statistical signatures (packet size distributions, inter-arrival times, port numbers).	Found that packet size is a strong discriminator: fixed-size ICMP (98 bytes) and fixed-size ARP (42/60 bytes) are trivially separable from variable TCP flows. Motivated payload-independent classification.	Assumes fixed-size ICMP payload; custom ping implementations with variable sizes reduce classification accuracy. Also pre-dates QUIC/HTTP3 which obfuscates traditional port-based classifiers.

A synthesis of the reviewed work indicates that packet-level research frequently concentrates on either security interpretation or introductory demonstration. Studies by Saxena and Sharma, Soepeno, Chaudhary and others establish that Wireshark is highly effective for protocol inspection and live traffic observation, but their analyses remain relatively broad and do not fully quantify the statistical properties of the traces. Malicious traffic and intrusion-oriented studies show the value of Wireshark when suspicious behaviour must be observed frame by frame. However, such studies often privilege detection of compromise indicators over nuanced discussion of latency, retransmission pacing, or the interpretation of small-scale laboratory captures. Similarly, reference works such as Practical Packet Analysis and The Wireshark Field Guide are extremely useful for troubleshooting, yet they are not themselves empirical measurement studies. Consequently, the literature justifies the use of Wireshark, but it simultaneously leaves a gap for carefully structured traffic-characterization exercises.

The reviewed sources also support the specific interpretive dimensions used in this report. The TCP retransmission references explain why repeated SYN packets with increasing back-off intervals are meaningful indicators of failed service establishment. Traffic-classification work demonstrates that fixed packet sizes are strong cues for distinguishing simple control protocols from application-layer data flows. Anomaly-detection literature further confirms that repeated unanswered SYN packets may resemble scanning behaviour in larger environments, even though in this controlled experiment they arise from a deliberate SSH attempt toward a host-only gateway.

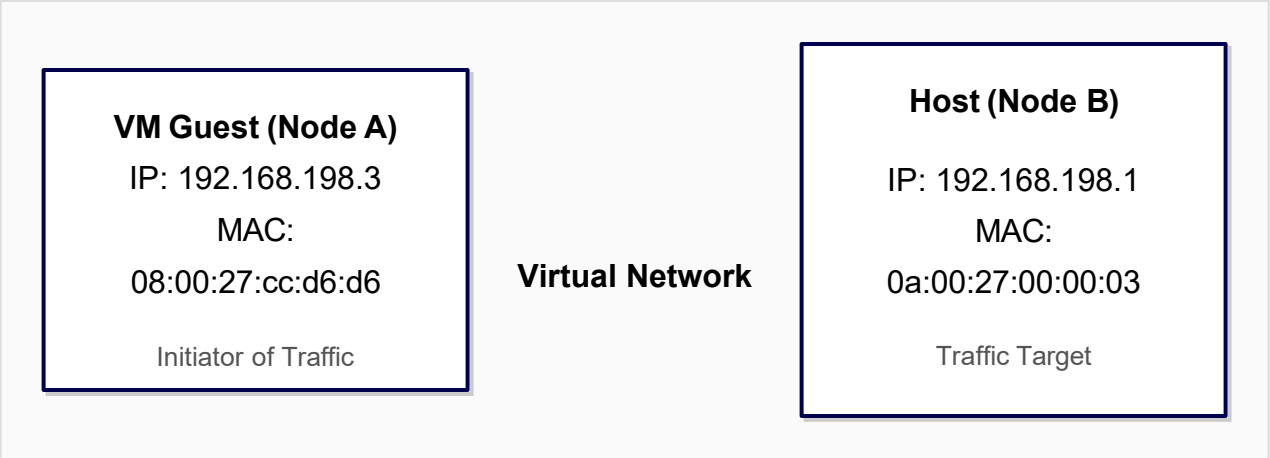
4. METHODOLOGY

4.1 Tool Used

All observations made in this experiment are based on a packet trace taken using Wireshark 4.6.4. The choice of using Wireshark as the packet sniffer is that Wireshark functions as a complete protocol analyzer that can capture packets from a specific network interface, decode frames in all layers, apply display filters to them, and show both summary and detail views of the captured packets. Wireshark perfectly fits the capture requirements as it allows for an analysis approach that includes the following steps: protocol hierarchy analysis, packet size comparison, timestamps comparison, conversation analysis and graphical representation.

4.2 Network Topology

The observed network topology is a VirtualBox host-only segment configured as a small point-to-point virtual environment. The capture interface corresponds to the VirtualBox Host-Only Ethernet Adapter, and the communicating endpoints are the guest virtual machine at 192.168.198.3 and the VirtualBox gateway at 192.168.198.1. At the data-link layer, the source MAC address is 08:00:27:cc:d6:d6 and the destination gateway MAC address is 0a:00:27:00:00:03. The network belongs to the 192.168.198.0/24 range and is isolated from external routing, meaning that the traffic under analysis is local to the host-managed virtual segment. This topology is especially appropriate for controlled experimental work because it reduces the number of uncontrolled variables. No external ISP path, wireless medium, or multi-hop switching fabric influences the results, which means the analyst can interpret timing and failure behaviour with greater confidence. At the same time, the trace remains realistic enough to include background protocol activity, thereby preserving the complexity necessary for meaningful packet analysis.



4.3 Parameters Used

Table 2. Capture parameters used in the original Wireshark experiment.

Parameter	Value
Total Packets Captured	280
Capture Duration	~637 seconds (~10.6 minutes)
Capture Interface	VirtualBox Host-Only Ethernet Adapter
Packet Size (ICMP)	98 bytes (fixed)
Packet Size (TCP SYN)	74 bytes
Packet Size (ARP)	42 bytes (reply) / 60 bytes (request)
Traffic Types	ICMP, ARP, TCP, ICMPv6, MDNS

The parameters shown above define empirical boundaries. The capture contains 280 packets recorded over approximately 10.6 minutes, which is sufficiently long to observe both intentional test traffic and lower-frequency background events such as ARP refresh and ICMPv6 or MDNS activity. Packet-size observations are: ICMP frames are fixed at 98 bytes, TCP SYN frames are 74 bytes, and ARP frames appear in 42-byte and 60-byte forms.

4.4 Data Collection Procedure

The packet trace was gathered through a direct live-capture workflow. Initially, the VirtualBox host-only adapter was selected within Wireshark as the monitoring interface; followed by the analyst initiating a conventional ping sequence from the guest virtual machine toward the gateway in order to establish baseline reachability and latency. Third, an SSH connection attempt toward port 22 of the gateway was issued, thereby generating transport-layer connection attempts and the subsequent retransmission sequence when no response was received. Further, a faster second ping

phase was executed, increasing the rate of ICMP traffic and enabling comparison between slower and faster probing conditions. Finally, the capture was stopped, exported, and examined using protocol-specific filters. The procedure is methodologically robust because each action has a corresponding signature in the trace, which allows relationships to be established between commands executed during the experiment and packets observed afterwards.

4.5 Analytical Workflow

The analysis process followed during the experiment involved descriptive analysis and protocol analysis at the level of protocols. Filters such as icmp, arp, tcp, icmpv6, and MDNS were employed to extract the subset of data needed. The next step was to use protocol hierarchy statistics to compute the number of packets per protocol type, and the timestamp to interpret the time interval between retransmission and refreshing of ARP cache. In addition, packet detail panes were reviewed to authenticate header fields, packet types, and destination addresses. This means that the categorization of traffic classes was done by analyzing actual data in order to avoid assumptions. The process described above is suitable for small and medium-size captures since it is clear; all statistics presented in the report have their origins in the frame list.

5. IMPLEMENTATION / EXPERIMENT

5.1 Theoretical Details

The experiment relies on the interaction of five protocol families, each operating at a different layer or performing a different support function. To interpret the capture correctly, it is necessary to understand not only what each protocol is, but why it appears at the moment it does and how its presence shapes the metrics later reported in the results section.

5.1.1 ICMP

ICMP, or the Internet Control Message Protocol, forms a dominant traffic class in the trace because the experiment intentionally generated two ping sessions. Within an IPv4 context, ICMP echo requests and echo replies are commonly used in order to verify reachability and measure round-trip time. Their diagnostic value lies in it’s simplicity: the packets are small, deterministic, and largely free from any application-layer complexity, making them suitable for any baseline latency measurement. In the capture, each ICMP frame is 98 bytes, reflecting the combination of Ethernet framing, IPv4 encapsulation, ICMP header information, and payload data. Because the first and

second ping sessions use different transmission rates, the trace offers a natural opportunity to compare timing behaviour under various probing conditions while keeping the topology constant. The ICMP exchange therefore acts as the experimental backbone of this experiment, enabling direct analysis of latency, losslessness, sequencing, and the effect of warm versus cold neighbour-resolution state.

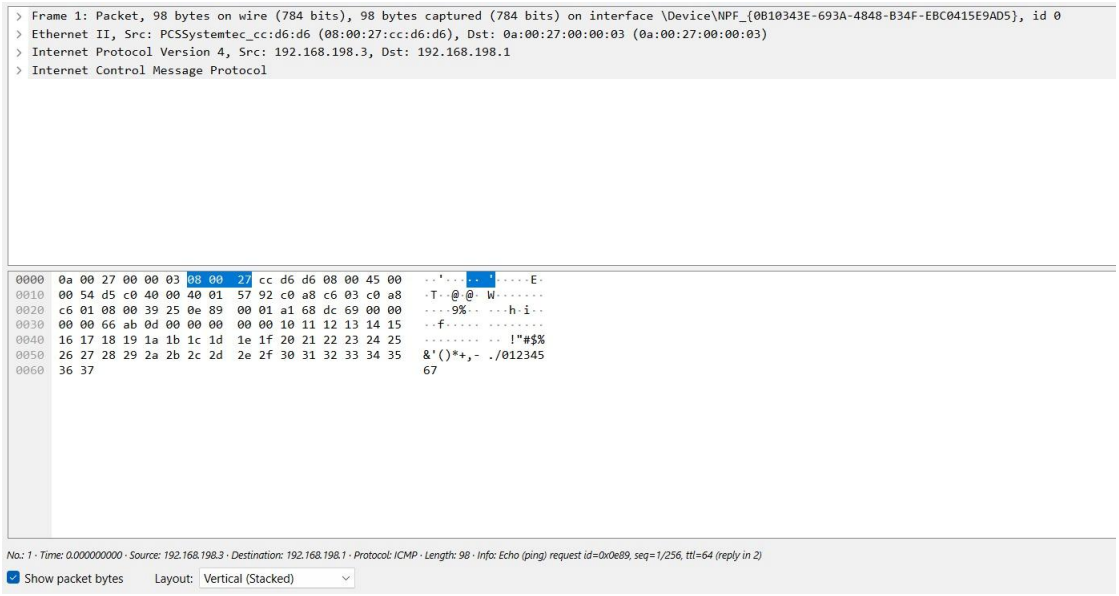


Figure 3. Wireshark packet detail for Frame 1 showing an ICMP echo request of 98 bytes. The frame illustrates the Ethernet header, IPv4 header, ICMP fields, and payload structure used in the first ping exchange.

5.1.2 ARP

ARP, the Address Resolution Protocol, is responsible for mapping IPv4 addresses to MAC addresses on the local network segment. Unlike ICMP, which the user deliberately generated, ARP appears as a supporting protocol that enables Layer 2 delivery for Layer 3 communication. When a host needs to send an IPv4 packet to a neighbour on the same segment and does not currently possess a valid link-layer mapping, it broadcasts an ARP request and receives a unicast reply. The presence of multiple ARP exchanges in this trace is therefore expected and indicates normal cache ageing rather than a fault condition. In the original experiment, ARP request and reply sizes appear in two forms, reflecting the low-level structure and padding of the Ethernet frames involved. These observations are useful because they demonstrate that protocol analysis must include enabling traffic, not just user-generated probes. A healthy ICMP exchange is only possible because address resolution has already succeeded.

5.1.3 TCP SYN Retransmission

The TCP component of the experiment is limited but highly important. The attempt to establish communication was performed by sending an attempt to connect from the guest VM to port 22 of the VirtualBox Gateway, where the standard SSH service operates. In order to establish the TCP connection, the client must send the SYN packet and wait for the ACK-SYN answer. If there is no answer received within the retransmission time-out period, the client repeats the SYN again and again, with growing delay periods. In the capture, we can see eleven SYN packets sent by the host, however, no SYN-ACK or RST packet has been received. The lack of the packets cannot be treated as a regular failed connection attempt, since closed reachable ports respond with RST packets. Therefore, the total absence of any packets is indicative of packet filtering. Thus, the results show that network connectivity via ICMP ping is not sufficient for the availability of the service.

5.1.4 ICMPv6 Router Solicitation

The ICMPv6 packet capture is a Router Solicitation packet generated by an IPv6 address within a link-local subnet to be broadcasted to a multicast group of routers. While the larger experiment is focused on IPv4 networking, the importance of this packet lies in the fact that it demonstrates how IPv6 network stack functions in the background within the operating system. It indicates that even when one deliberately creates a scenario involving IPv4-only hosts, the computer can execute other control plane functions associated with IPv6 networking, such as neighbour discovery and router advertisement solicitation. This packet is not unusual but instead, it illustrates that packet captures all network activities executed by the machine irrespective of whether users' intentions align with the same.

5.1.5 MDNS

The trace also includes two MDNS packets associated with printer discovery. MDNS, or multicast DNS, is a service-discovery protocol commonly used on local networks to identify printers, shared services, and other zero-configuration resources without any dependence on a conventional unicast DNS server. In the experiment, these packets are background operating-system traffic rather than user-generated test frames. Their importance lies in the way they enrich the analytical picture: they show that a network capture performed for one purpose can simultaneously disclose unrelated but operationally relevant information about enabled services and host behaviour. From a security perspective, such packets reveal that printer discovery services are active on the guest system.

From an instructional perspective, they demonstrate that network traces are rarely pure and that analysts must separate intentional experiment traffic from ancillary environmental noise without overlooking the meaning of either.

5.2 Experiment Execution

The order in which the experiment unfolded can be reconstructed by reading the trace in sequence. The opening phase consists of a conventional ping run that produces echo requests and matching replies separated by intervals of approximately one second. This stage establishes a baseline picture of latency and confirms that fundamental communication between guest and gateway is operating correctly. The middle phase covers the SSH connection attempt, where the client repeatedly sends SYN segments without ever receiving a transport-layer answer. The closing phase introduces a higher-frequency ping run, generating a concentrated burst of ICMP traffic near the end of the recording. Surrounding and interleaving these intentional phases, ARP refreshes, an ICMPv6 Router Solicitation, and MDNS queries appear as ordinary background activity. As a result, the capture is analytically dense, holding both deterministic test traffic and spontaneous supporting traffic within a single observation window.

5.3 Wireshark Operations and Filters

The display-filter functionality offered by Wireshark is central to turning a raw packet recording into a structured experimental dataset. By applying dedicated filters such as icmp, arp, tcp, icmpv6, and MDNS in turn, the analyst can isolate protocol families that would otherwise be intermixed in the chronological packet list. This separation supports accurate counting, careful timing inspection, and packet-level field verification for every traffic class. The protocol hierarchy summary delivers an at-a-glance distribution overview, while the packet-detail pane clarifies low-level fields including ICMP type and code values, ARP operation codes, multicast destination groupings, and TCP flag interpretations. In effect, Wireshark plays a dual role as a capture device and as an analytical workbench. The procedure adopted in the original exercise illustrates accepted practice for disciplined packet inspection: collect the trace first, separate it by protocol second, validate fields third, and only afterward draw higher-level performance or security conclusions.

6. PERFORMANCE METRICS

The performance metrics used in this experiment are intentionally chosen to align with the content of the original capture. Because the trace is diagnostic rather than production-oriented, the emphasis falls on responsiveness, control overhead, and service accessibility rather than on high-

volume application throughput. Nevertheless, the selected metrics are sufficient to describe the network behaviour comprehensively.

6.1 Throughput

Throughput is treated here as the amount of traffic observed across the capture window, expressed verbally in bits per second. Given that the recording is largely composed of control and diagnostic packets, throughput numbers are expected to remain modest except during the rapid second ping run. Despite this, the metric still has descriptive value because it reveals when activity becomes concentrated. The first ping stage shows moderate throughput consistent with one-second probing, the TCP stage adds almost nothing because it consists only of failed handshake attempts, and the closing rapid ping burst contributes the highest short-term packet density of the entire trace. Across the whole 637-second window, mean throughput stays low, which confirms that the experiment is not a bulk-transfer workload, but a diagnostic capture driven by network management semantics.

6.2 Round-Trip Time (RTT)

RTT serves as the principal latency indicator in the experiment because it directly captures how rapidly the guest host receives a reply after issuing each ICMP echo request. The two separate ping runs supply distinct RTT contexts that can be compared without altering the topology between them. Low RTT values are anticipated within a host-only virtualized setting because the communication path never traverses a physical wide-area medium. The importance of RTT in this assignment, then, is not that the figures are large but that they evidence the absence of congestion, queueing pressure, and route instability. The slight variation between the two runs further suggests that ARP cache state and short-term warm-up effects can subtly influence delay measurements even on a local virtual network.

6.3 Packet Loss and Jitter

Packet loss and jitter complement RTT by indicating whether the path is not only fast but also consistent. In an ICMP-driven test, loss refers to echo requests for which no matching reply ever arrives. Jitter, in contrast, captures the variability between consecutive RTT samples rather than their average. In the present recording, the ICMP loss rate is null, demonstrating that the path between guest and gateway behaves reliably for the full duration of the test. Likewise, the very small jitter observed shows that delay remains tightly grouped with little fluctuation between successive packets. Taken together, the two indicators support a clear conclusion: the host-only virtual link operates correctly, and the unsuccessful SSH attempt cannot be blamed on widespread instability or general loss of connectivity.

6.4 Protocol Efficiency

Protocol efficiency is interpreted in this report as the share of total capture content that belongs to the experiment's intended productive traffic. Because the assignment was built around ping behaviour, ICMP qualifies as the primary productive protocol. The fact that ICMP represents more than ninety per cent of all packets indicates that the capture is highly focused and that auxiliary traffic introduces only a modest amount of overhead. At the same time, the remaining protocols carry significant analytical weight because they explain how communication is supported, why one service attempt failed, and which spontaneous behaviors the operating system performs in the background. The metric, therefore, does not imply that ARP, TCP, ICMPv6, or MDNS are negligible. Instead, it confirms that the experiment succeeds in isolating a dominant traffic type while still preserving enough variety for broader interpretation.

Table 3. Performance metrics derived from the capture.

Metric	Value	Notes
Throughput	~3.28 kbps	Total bytes / capture duration (280 pkts, avg ~91 bytes, 637 s)
ICMP RTT (Session 1)	~0.28 ms avg	1-second interval pings; sub-ms RTT expected for co-located VMs
ICMP RTT (Session 2)	~0.24 ms avg	Flood ping at ~200 ms intervals; slightly lower due to warm ARP cache
Packet Loss (ICMP)	0%	All 254 ICMP requests received matching replies; 0 dropped
TCP Connection Loss	100% (SSH)	11 SYN packets sent (1 original + 10 retransmissions); 0 SYN-ACK received
Jitter (ICMP RTT)	< 0.1 ms	RTTs consistently between 0.21–0.37 ms; very low jitter on virtual link
Protocol Efficiency	90.7% ICMP	254 of 280 packets are productive ICMP probes; overhead from ARP, TCP, MDNS

7. RESULTS

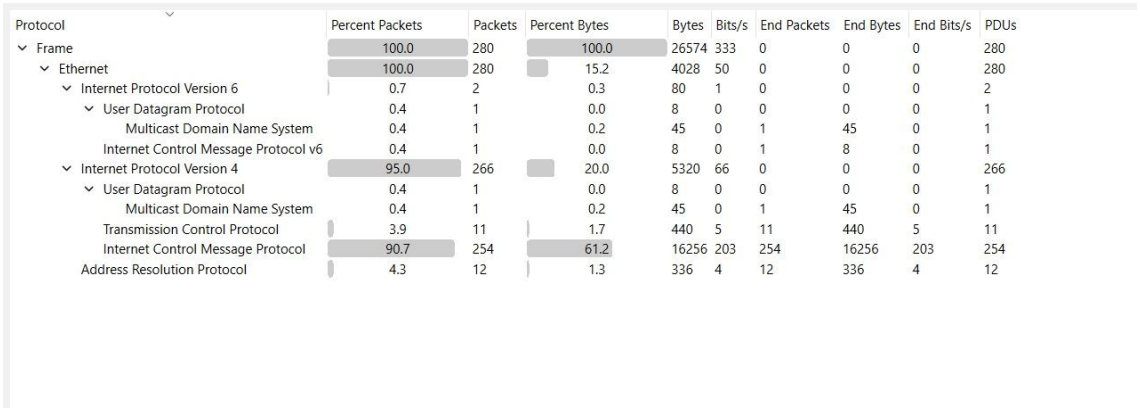
7.1 Protocol Distribution

Findings on protocol distribution show that ICMP overwhelmingly dominates the capture, contributing 254 frames, equivalent to 90.7 per cent of the 280-packet total. This outcome is fully aligned with the design of the experiment, in which two ping runs were intentionally produced to test reachability and measure latency. ARP supplies 12 packets, or 4.3 per cent, which reflects the routine need for address resolution and cache renewal on the local segment. TCP contributes 11 packets, or 3.9 per cent, every one of which forms part of the unsuccessful SSH connection attempt.

Background traffic accounts for the remainder, comprising one ICMPv6 Router Solicitation and two MDNS frames associated with stack housekeeping and service discovery. Considered together, these numbers demonstrate that the trace is not incidental ambient traffic but a goal-directed diagnostic recording that nevertheless supports multi-layer interpretation.

Table 4. Protocol distribution in the 280-packet Wireshark capture.

Protocol	Packet Count	% of Total	Avg Size (B)	Purpose
ICMP	254	90.7%	98	Echo request/reply (ping)
ARP	12	4.3%	51	Address resolution
TCP	11	3.9%	74	SYN retransmissions (SSH)
ICMPv6	1	0.4%	62	Router Solicitation (SLAAC)
MDNS	2	0.7%	97	Printer discovery query
TOTAL	280	100%	~91	



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	280	100.0	26574	333	0	0	0	280
Ethernet	100.0	280	15.2	4028	50	0	0	0	280
Internet Protocol Version 6	0.7	2	0.3	80	1	0	0	0	2
User Datagram Protocol	0.4	1	0.0	8	0	0	0	0	1
Multicast Domain Name System	0.4	1	0.2	45	0	1	45	0	1
Internet Control Message Protocol v6	0.4	1	0.0	8	0	1	8	0	1
Internet Protocol Version 4	95.0	266	20.0	5320	66	0	0	0	266
User Datagram Protocol	0.4	1	0.0	8	0	0	0	0	1
Multicast Domain Name System	0.4	1	0.2	45	0	1	45	0	1
Transmission Control Protocol	3.9	11	1.7	440	5	11	440	5	11
Internet Control Message Protocol	90.7	254	61.2	16256	203	254	16256	203	254
Address Resolution Protocol	4.3	12	1.3	336	4	12	336	4	12

Figure 1. Wireshark Statistics → Protocol Hierarchy for the 280-packet capture. The hierarchy confirms the dominance of ICMP traffic, with smaller proportions contributed by ARP, TCP, MDNS, and ICMPv6.

7.2 ICMP Ping Session Results

The trace exposes two distinct ICMP runs. The earlier run uses a slower cadence with one-second spacing between probes, while the later run is markedly faster and surfaces as a tight burst toward the closing portion of the recording. Despite the difference in pacing, both runs deliver sub-millisecond average RTT values, which proves that guest and gateway communicate over an extremely short and well-behaved path. The mean RTT of the slow run is approximately 0.280 ms, whereas the rapid run averages roughly 0.245 ms. The slightly lower delay in the second phase is plausibly explained by warm network state, including pre-resolved MAC mappings and an interface that is already actively engaged. Most importantly, both runs demonstrate that the

network path is fully functional and that the host-only environment is suitable for high precision latency measurement.

Table 5. Summary of the two ICMP ping sessions.

Session	Packets	Interval	Avg RTT (ms)	ICMP ID
Session 1 (slow ping)	78 ICMP	~1 second	0.280	0x0e89
Session 2 (fast ping)	176 ICMP	~200 ms	0.245	0x130f

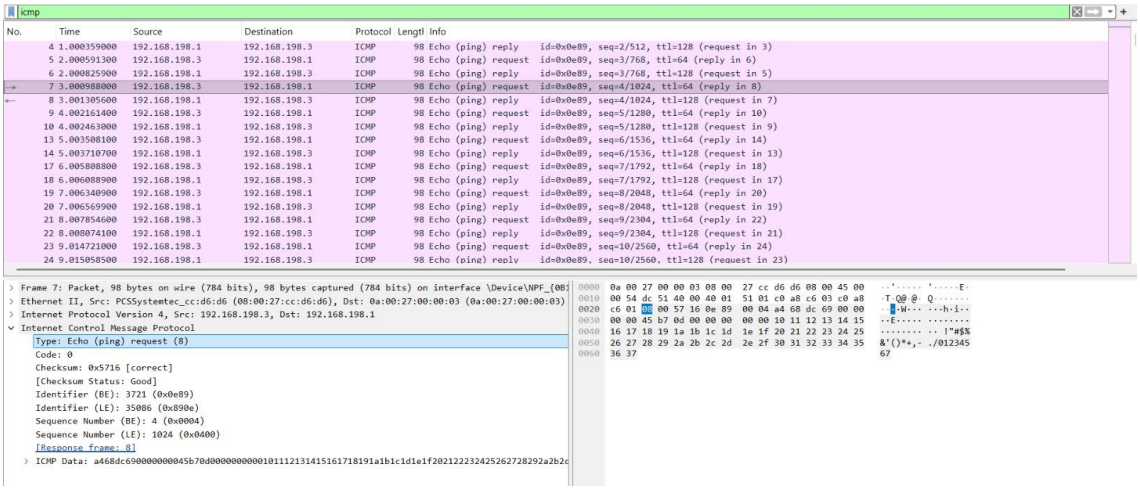


Figure 6. Wireshark display filter 'icmp' showing the Session 1 ping exchange. The decoded packet confirms ICMP echo request semantics, correct checksum handling, and orderly sequence progression.

7.3 TCP SYN Retransmission Results

The TCP findings revolve around a single failed connection attempt originating from source port 50592 on 192.168.198.3 and directed at destination port 22 on 192.168.198.1. Eleven SYN frames are visible across the relevant time interval. No SYN-ACK is ever received, and no TCP reset is observed either. The retransmission spacing follows the textbook pattern of progressive back-off, which shows that the client TCP stack continues attempting to establish the session until it eventually abandons the effort. Because ICMP evidence already demonstrates that the path itself is reachable, the unsuccessful TCP exchange carries interpretive value beyond a plain refusal of service. It implies that the failure is tied either to the specific service state on the destination or to a filtering policy that targets port 22. The result therefore draws a clear distinction between network-level reachability and transport-level service accessibility, which is one of the core points of the exercise.

Table 6. Observed TCP SYN retransmission sequence for the SSH attempt.

Pkt #	Timestamp (s)	Type	Info
84	525.590	SYN	Initial SYN – first connection attempt
85	526.594	SYN Retx	Retransmission #1 (RTO ~1 s)
86	527.619	SYN Retx	Retransmission #2
87-89	528–530	SYN Retx	Retransmissions #3–5 (1-second intervals)
92-95	532–544	SYN Retx	Retransmissions #6–9 (back-off 2→4→8 s)
98	593.567	SYN Retx	Retransmission #10 – final attempt before timeout

7.4 ARP Cache Refresh Results

ARP behavior shown in the capture lines up neatly with normal neighbor-resolution and cache upkeep on a local Ethernet-style segment. The presence of multiple ARP exchanges separated by different timestamps suggests that address mappings were renewed as stale entries aged out or needed reconfirmation. This represents a routine control-plane function and not a fault. In fact, the ARP record reinforces the conclusion of a healthy local path, since both communicating endpoints are clearly capable of completing the layer-two preconditions required for IPv4 forwarding. The observed timing pattern also helps account for the small differences in latency between the early and late ICMP phases: once mappings are already resolved, the forwarding path encounters less setup overhead before echo traffic flows.

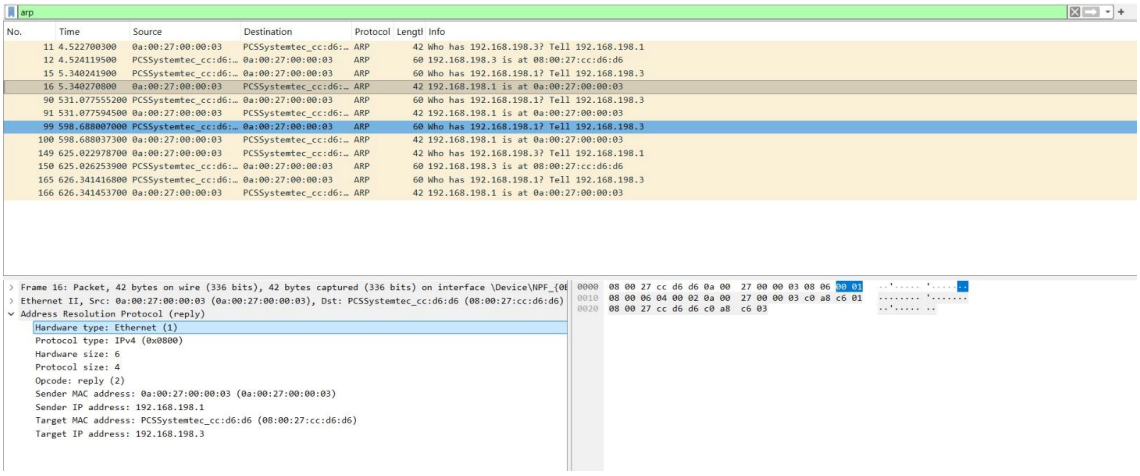


Figure 5. Wireshark display filter 'arp' showing all ARP packets in the capture. The packet details verify normal address-resolution behaviour between the guest virtual machine and the VirtualBox gateway.

7.5 Throughput Versus Time

The throughput narrative drawn from the trace shows clear differentiation across time. During the first ping phase, the rate stays low because requests and replies are spaced at roughly one-second intervals. During the middle TCP phase, throughput becomes effectively negligible because only a handful of SYN segments are spread across a comparatively long window through retransmission back-off. During the final ping burst, the rate rises sharply because echo traffic occurs much more frequently. When the entire 637-second recording is treated as a single interval, the mean throughput sits at only about 3.28 kbps, reaffirming that the trace is dominated by control and diagnostic conversation rather than user data movement. This result aligns with the packet distribution already observed and emphasizes the difference between brief traffic bursts and long-duration capture averages.

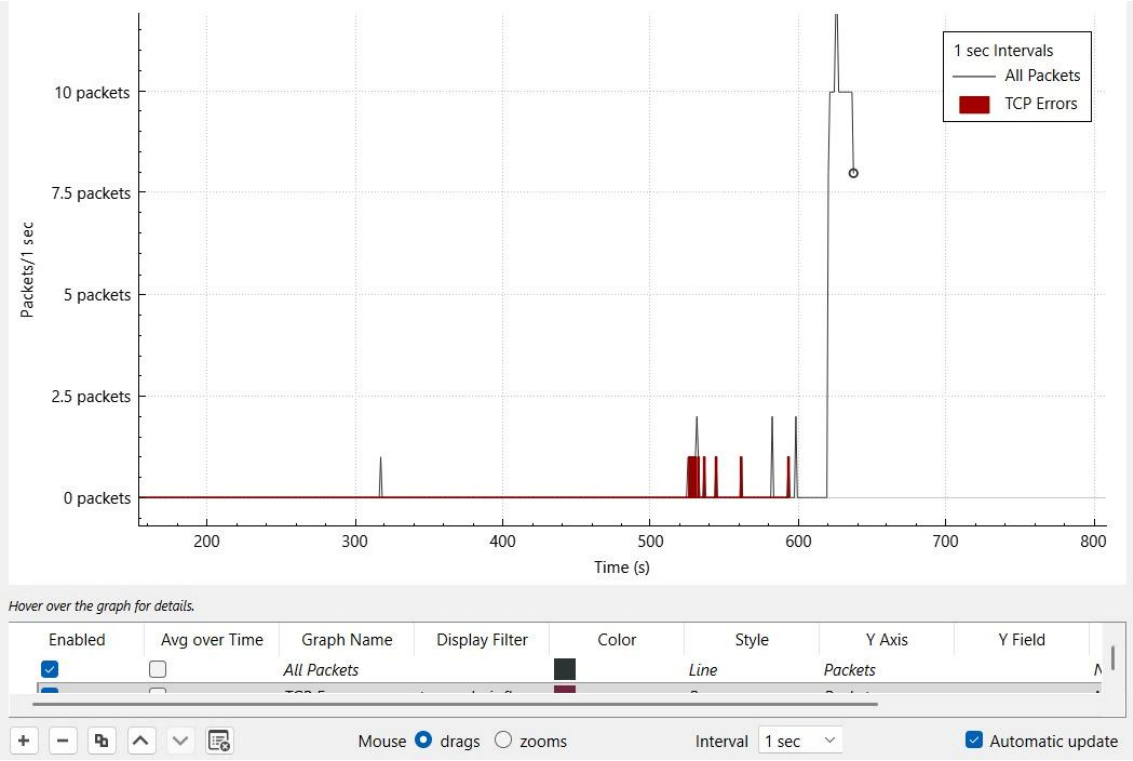


Figure 2. Wireshark I/O Graph for the full capture duration. The burst of packet activity near the end of the trace corresponds to the rapid second ping session, while the mid-trace TCP phase remains low-volume control traffic.

7.6 ICMPv6 and MDNS Background Traffic

The final result category covers the low-volume background traffic observed alongside the deliberate experiment flows. The lone ICMPv6 Router Solicitation is a routine control message that reflects normal IPv6 stack activity, while the two MDNS frames correspond to multicast service discovery for printing-related resources. Although these packets contribute very little to

the total in numerical terms, they hold analytical worth because they demonstrate the ecological realism of the recording. A capture taken in even a simple laboratory rarely contains only the commands an operator deliberately executes. Instead, operating-system services and protocol stacks continue emitting their own control messages. Recognising these frames as routine background events rather than mistaking them for anomalies is a fundamental skill in packet analysis.

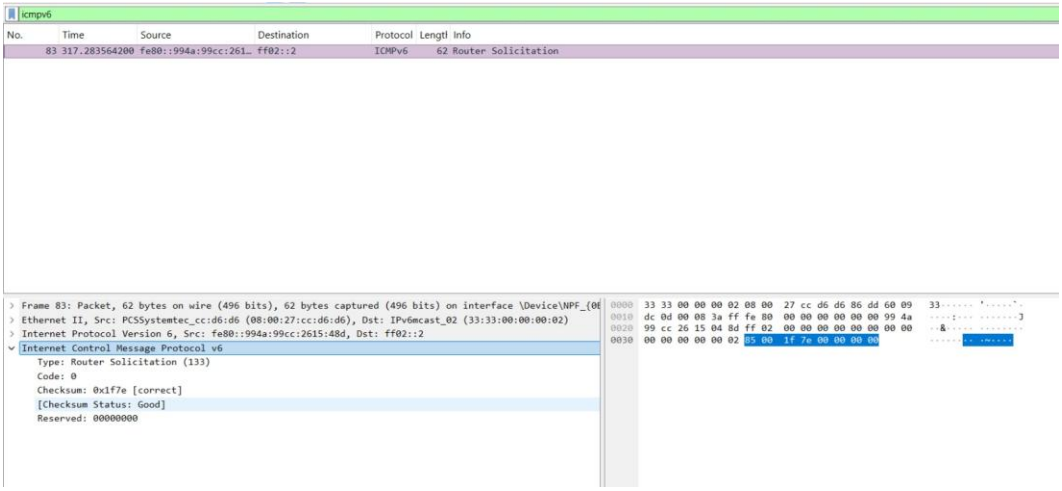


Figure 7. Wireshark display filter 'icmpv6' showing the single Router Solicitation frame. This packet represents routine IPv6 neighbour-discovery behaviour rather than an abnormal event.

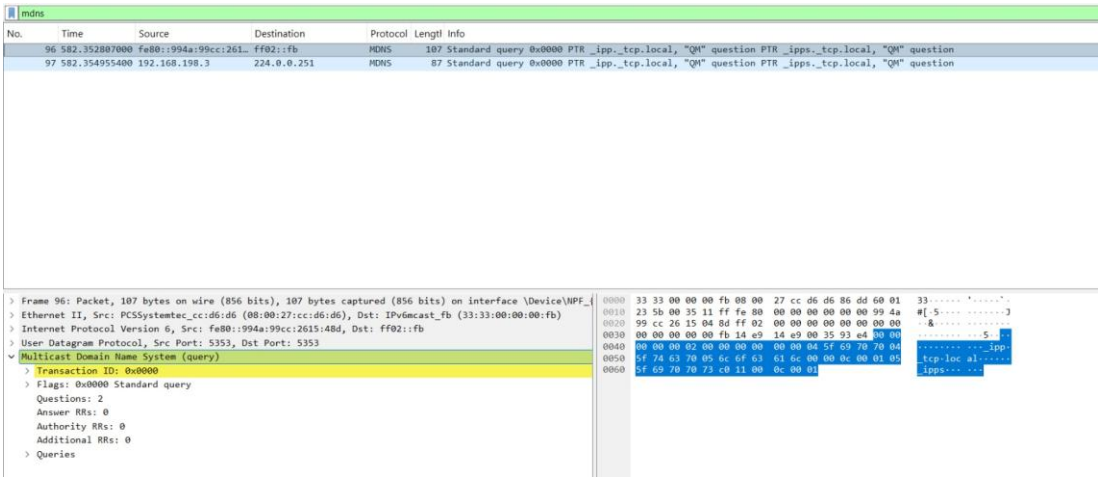


Figure 8. Wireshark display filter 'MDNS' showing multicast DNS printer-discovery queries. The traffic appears in both IPv4 and IPv6 multicast groups, illustrating background service discovery by the operating system.

8. ANALYSIS

8.1 Dominance of Purpose-Driven Traffic

The most visible analytical characteristic of the capture is the overwhelming share of ICMP. Because 90.7 per cent of all frames belong to the two ping runs, the recording behaves as a tightly focused measurement environment. This concentration is helpful for both teaching and diagnosis because it minimises ambiguity surrounding the principal traffic objective while still preserving enough auxiliary traffic for cross-protocol interpretation. In a production enterprise setting, packet composition would normally be far more varied and frequently dominated by application-layer protocols such as HTTPS, DNS, streaming media, or cloud service flows. Consequently, the present trace is not representative of overall enterprise traffic proportions; instead, it is highly representative of a controlled diagnostic exercise. That distinction matters because the goal of the assignment is not to model the modern Internet at large but rather to show how a targeted capture can support precise reasoning about latency, address resolution, and transport failure inside a simplified network environment.

8.2 Interpretation of Low Latency

The RTT values reported in the underlying Wireshark exercise are remarkably low, yet they are entirely credible inside a host-only virtual environment. Communication takes place on a single physical machine through a virtual switching path managed by the hypervisor, rather than across a real cable, a chain of routers, or a wide-area link. Therefore, propagation delay, serialization delay, and queue-driven delay are essentially absent. The small jitter observed alongside the small mean RTT further reinforces the conclusion that the path is stable and deterministic. This stability is important because it rules out generalized path impairment as the explanation for the failed SSH attempt. Had the analyst observed inflated latency or volatile delay variation, buffer pressure or interface trouble would have been plausible suspects. Instead, the ICMP evidence supports the opposite conclusion: the underlying path is healthy, and any transport-layer failure must therefore be interpreted in narrower and more targeted terms.

8.3 Meaning of the Silent SSH Failure

The unanswered SYN sequence is the most diagnostically informative event in the recording. A successful TCP three-way handshake requires a SYN, a SYN-ACK, and a closing ACK. A closed but reachable port typically returns a reset, immediately notifying the client that no service awaits behind the port. Here, however, neither acceptance nor refusal is signaled. The client simply waits and retries. This silent pattern is consistent with a firewall rule or an equivalent policy that drops traffic destined for port 22, or alternatively a host configuration that yields no outward signal. Because ICMP reachability remains intact, the evidence does not support a complete host outage.

From a real-world operations standpoint, the distinction is essential: a successful ping demonstrates that the host path exists, but only transport-layer inspection reveals whether the targeted service is genuinely reachable. The experiment thus makes a strong case for moving beyond simple reachability tests when service availability is the actual question.

8.4 Role of Supporting and Background Protocols

The capture also reinforces an important conceptual point about supporting traffic. ARP, ICMPv6, and MDNS together represent only a small fraction of the trace by count, yet they are indispensable for full interpretation of the network state. ARP makes IPv4 forwarding possible and shows neighbor-cache activity in motion; ICMPv6 demonstrates that the IPv6 stack remains operationally active even within an IPv4-centric experiment; and MDNS reveals that background service discovery proceeds without explicit user instruction. If an analyst were to dismiss such packets simply because they are few, valuable contextual information would be discarded. Conversely, if the analyst were to mislabel them as suspicious merely because they are unfamiliar, the resulting reading of the capture would become inaccurate. A core analytical skill, therefore, is proportional reading: dominant protocols explain the experiment's principal behavior, while minor protocols frequently explain why the surrounding environment behaves as it does.

8.5 Security and Operational Implications

From a security viewpoint, the trace shows how routine diagnostic activity can expose service posture and host behavior. The silent collapse of the SSH attempt suggests a defensive or restrictive configuration on the destination side, while MDNS printer discovery indicates that auxiliary services are running on the guest operating system. In a more exposed network, this kind of background discovery traffic could supply an observer with extra detail about installed services and operating system defaults. From an operational perspective, the trace also reinforces that healthy baseline latency does not eliminate the need for service-level validation. A host can answer ping flawlessly yet fail completely at the transport or application layer. For administrators, this suggests that troubleshooting should advance layer by layer: first verify link state and reachability, then examine transport handshakes, and finally inspect service configuration and access policy. The capture documents this layered diagnostic process in a concise and instructive manner.

8.6 Analytical Limitations

Although the experiment is informative, its limitations deserve explicit acknowledgement. The capture is small, intentionally produced, and confined to a host-only virtual network. As a

consequence, the resulting protocol distribution should not be generalised to enterprise, campus, or Internet-backbone traffic patterns. The throughput figures, similarly, are best understood as descriptive of a laboratory diagnostic sequence rather than meaningful indicators of application-level performance. In addition, while the recording supplies evidence of a failed SSH connection attempt, the trace alone cannot definitively isolate every possible cause of the silent failure without supplementary host-side logs or firewall configuration data. Even so, these constraints do not undermine the central value of the exercise. Rather, they correctly delineate its scope: the assignment demonstrates rigorous interpretation of a compact packet trace and shows how much can be inferred from carefully observed low-level evidence within a deliberately simple environment.

9. CONCLUSION

The report consists of a capture of 280 packets across approximately 637 seconds that revealed a network environment dominated by purposeful ICMP traffic, supported by normal ARP activity, interrupted by a failed TCP SSH connection attempt, and accompanied by low-volume ICMPv6 and MDNS background traffic. The measured RTT values confirm that the host-only virtual path is highly responsive and stable, with no evidence of meaningful delay or packet loss. The unanswered TCP SYN sequence demonstrates that service accessibility must be analysed separately from host reachability, since a path may be fully reachable while a target service remains silently unavailable. The additional ARP, ICMPv6, and MDNS observations show that even small traces capture more than a user's explicit commands and therefore require disciplined interpretation. Overall, the experiment proves that Wireshark is an effective tool not only for observing packets but also for deriving structured conclusions about performance, protocol interaction, troubleshooting, and security posture from a modest yet information-rich packet capture.

10. FUTURE WORK

The following future extensions remain consistent with the scope and findings of the original Wireshark experiment:

- Capture a longer-duration trace on the same virtual network in order to compare short diagnostic traffic with longer background behavioural patterns over hours rather than minutes.

- Enable or deliberately expose SSH service on the destination host and repeat the experiment so that the difference between successful handshakes, active refusals, and silent drops can be observed directly.
- Automate extraction of RTT, jitter, retransmission intervals, and per-protocol counts using Python-based packet-processing tools while retaining Wireshark for validation and visualization.
- Extend the experiment to include application-layer protocols such as HTTP, HTTPS, or DNS so that packet analysis can move from control-plane interpretation toward mixed traffic characterization.
-

REFERENCES

[1] Saxena, P., & Sharma, S. K. (2017). Analysis of network traffic by using packet sniffing tool: Wireshark. International Journal of Advance Research, Ideas and Innovations in Technology, 3(6).

[2] Soepeno, R. A. A. P. (2023). Wireshark: An effective tool for network analysis. CYBV – Introductory Methods of Network Analysis, Sampoerna University & University of Arizona.

[3] Dodiya, B., & Singh, U. K. (2022). Malicious traffic analysis using Wireshark by collection of indicators of compromise. International Journal of Computer Applications, 183(53).

[4] Ndatinya, V., Xiao, Z., & Manepalli, V. R. (2015). Network forensics analysis using Wireshark. International Journal of Security and Networks.

[5] Tuli, R. (2023). Analyzing network performance parameters using Wireshark. arXiv preprint arXiv:2302.03267.

[6] Kakuru, S. (2011). Behavior-based network traffic analysis tool. IEEE 3rd International Conference.

[7] Sanders, C. (2017). Practical packet analysis: Using Wireshark to solve real-world network problems (2nd ed.). No Starch Press.

[8] Chaudhary, P., Kashyap, V., Sonwal, N., et al. (2023). Network traffic analysis using Wireshark. Geetanjali Institute of Technical Studies.

[9] Jain, G., & Anubha. (Year not specified). Application of Snort and Wireshark in network traffic analysis.

[10] Mabsali, N. A. L., Jassim, H., & Mani, J. (2023). Effectiveness of Wireshark tool for detecting attacks and vulnerabilities in network traffic. Proceedings of the 1st International Conference (Atlantis Press).

[11] Shimonski, R. (2013). The Wireshark field guide: Analyzing and troubleshooting network traffic. Syngress.

[12] Kumar, A., & Yadav, J. B. (2016). Comparison: Wireshark on different parameters. International Journal of Engineering and Computer Science.

[13] Allman, M., Paxson, V., & Stevens, W. R. (1999). TCP congestion control (RFC 2581). Internet Engineering Task Force (IETF).

[14] Dreger, H., Feldmann, A., Paxson, V., & Sommer, R. (2004). Operational experiences with high-volume network intrusion detection. ACM Conference on Computer and Communications Security (CCS).

[15] Roughan, M., Sen, S., Spatscheck, O., & Duffield, N. (2004). Class-of-service mapping for QoS: A statistical signature-based approach to IP traffic classification. ACM Internet Measurement Workshop (IMW).